University of Bucharest
Faculty of Mathematics and Computer Science

Documentation Project PML

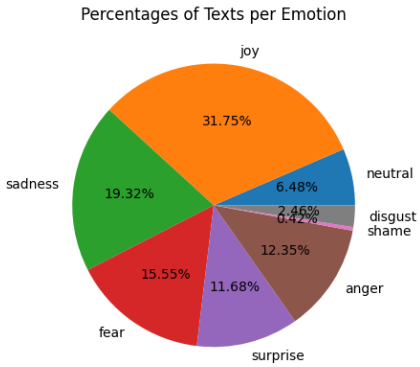
Capatina Razvan Nicolae, group 407

Dataset Details

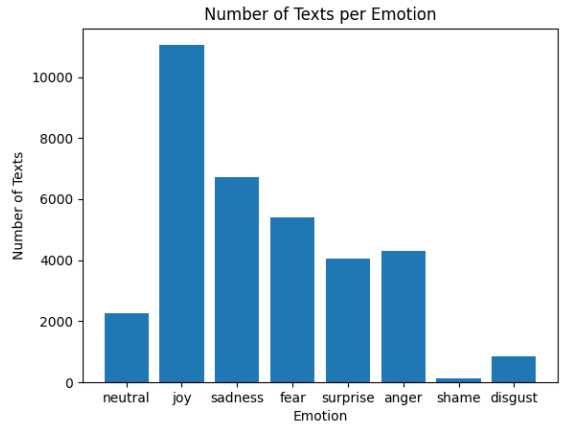
The used dataset is Text with Sentiment, available on Kaggle[1]. The data consists of 34.792 samples, under the form of a natural language text written in English and a label, denoting the sentiment of the writer in that sample. There are in total 8 classes: anger, disgust, fear, joy, neutral, sadness, shame and surprise. The task is to classify each text sample to the sentiment class it belongs to.

Before using the dataset, the first step was to make sure there are no empty text samples or samples consisting only of empty spaces. Also, there must be no missing labels for any text and all labels should belong to the set of sentiments mentioned previously.

In figures 1a and 1b we can see the composition of the dataset and its unbalanced nature.



(a) Dataset Composition in Percentages



(b) Dataset Composition in Numbers

The next step was to pre-process the data so that each text is simplified in order to more accurately identify the differences between classes. Therefore, each text has any leading or trailing spaces removed, all characters were converted to lowercase and only letters and spaces between words were kept (digits or numbers were also removed).

Afterwards, all stop-words were removed. Here, the English language stop-words set provided by Natural Language ToolKit (NLTK) package in Python has been used.

Now we can visualize 2 for each sentiment the average number of meaningful words in all samples belonging to that class.

[1]Dataset Kaggle Link: `https://www.kaggle.com/datasets/divu2001/text-with-sentiment`

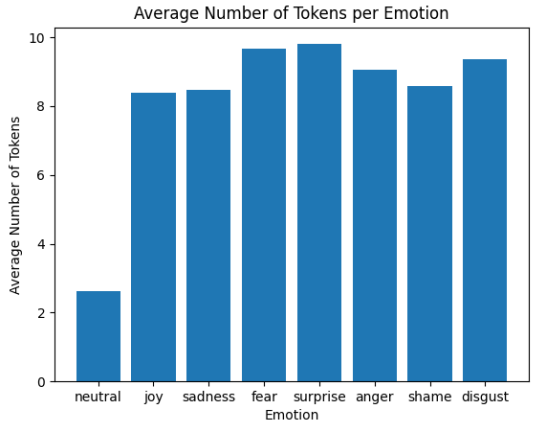


Figure 2: Average Number of Words per Sentiment

An interesting observation is that neutral texts have only around 2-3 words per sample, while all emotional texts tend to be longer, around 8-9-10 words per sample.

Below is a table 1 showing which are the top 10 most used words per sentiment (along with their frequencies in the samples).

Table 1: Top 10 Most Used Words per Sentiment

	neutral	joy	sadness	fear	surprise	anger	shame	disgust
1.	yes (155)	day (794)	im (574)	afraid (577)	en (335)	angry (318)	embarrassed (64)	like (92)
2.	see (77)	time (705)	sad (424)	fear (492)	een (307)	one (251)	ashamed (35)	people (78)
3.	oh (68)	im (697)	time (336)	love (487)	im (293)	anger (224)	embarrassment (28)	dont (67)
4.	ok (65)	amp (682)	like (320)	today (408)	de (260)	dont (223)	nt (9)	im (56)
5.	like (64)	christmas (622)	dont (319)	tomorrow (405)	home (176)	time (208)	mortified (8)	amp(50)
6.	think (58)	love (593)	one (305)	seen (364)	know (169)	people (208)	humiliated (8)	hate (44)
7.	sure (58)	work (556)	get (261)	yesterday (354)	one (167)	get (205)	felt (8)	rt (40)
8.	know (49)	today (524)	day (258)	one (303)	ik (164)	friend (200)	see (6)	disgruntled (36)
9.	right (49)	get (514)	cant (249)	im (293)	day (164)	im (178)	get (6)	u (33)
10.	good (47)	like (497)	died (235)	dont (292)	rt (164)	like (168)	rather (6)	get (32)

Feature Extraction Methods

The following two approaches for extracting features have been applied on the pre-processed texts:

- Term Frequency-Inverse Document Frequency (TF-IDF), which was set to output a representation of 128 features for each sample of text.
- A pre-trained fasttext model (cc.en.300.bin, available at [2]), which outputted embeddings of 300 features for each word in a text. The mean of all word embeddings from a single sample has been used in order to obtain a 300-dimensional embedding for a single text. Fasttext has been pre-trained in an unsupervised manner on Common Crawl [3], a raw web scrape, containing billions of automatically collected webpages.

A train-validation split percentage of 70% with stratification has been used, in order to split the resulted datasets in a train and a validation part. Stratification ensures that both train and validation sets have same distribution of labels.

[2] cc.en.300.bin fasttext Model Download Link: `https://fasttext.cc/docs/en/crawl-vectors.html`
[3] Common Crawl: `https://commoncrawl.org/`

Unsupervised Methods

The following two unsupervised methods have been used:

- BIRCH (Balanced Iterative Reducing and Clustering using Hierarchies), which works by incrementally building a hierarchy of clusters (under the form of a search tree).
  - Each node is either an internal node or a leaf node.
  - An internal node contains at most branching_factor children (each child having an associated clustering feature, CF).
  - A leaf node contains at most branching_factor clustering features (CFs), each being associated with a micro-cluster, the smallest unit of clusters.
  - When a new sample is inserted, it updates the micro-cluster it belongs to (by traveling the tree from root to leaf and updating the associated CF) and also updates all intermediary metadata from leaf to root.
  - If at any point in the tree, the number of children or the number of micro-clusters of a node exceeds branching_factor, then a split occurs, creating new nodes.
  - A clustering feature (CF) is a triple (N, LS, SS), where N is the number of nodes in that cluster, LS is the sum (component-wise) of all data points in that cluster and SS is the sum of squared points (also computed component-wise) of all points from cluster.
  - By having only (N, LS, SS), we can compute useful information, such as the centroid of a cluster ($\mu = \frac{LS}{N}$).
  - There is also a maximum admitted threshold for the size of each cluster, which is used as a hyperparameter, besides the already mentioned branching_factor (clusters are not allowed to go past this threshold value, a split will occur in that case in order to restore the constraint).
  - The whole method is efficient in terms of memory, since it only stores the clustering features, not the samples themselves. Also, in terms of time complexity, each sample takes logarithmic time to be inserted into the tree.
  - One issue of the method is that it can generate very different hierarchies of clusters depending on the order in which the samples have been iteratively inserted into the structure (random shuffling the data before fitting the model is useful).
- Fuzzy C-Means
  - It is the soft version of K-Means, which instead of assigning each point a cluster it belongs to, it assigns a simplex (for each point, we have the probabilities of that point belonging to each existing cluster).
  - In order to obtain an exact cluster the point belongs to, we can use an argmax function on the computed probabilities.
  - Similarly to K-Means, the method does multiple iterations, updating the cluster membership probabilities until they converge (the change between consecutive iterations is below an error fixed as parameter).
  - The method has a tunable hyperparameter, the fuzziness exponent (written as m), which controls how soft will the membership probabilities be for each sample. If $m \to 1$, then the method approaches hard K-Means. If m is big, then the probabilities become more and more uniform (softer). If $m \to \infty$, then all probabilities will converge towards $\frac{1}{c}$, where c is the number of clusters we want to obtain.

Hyperparameter Tuning of Supervised Baseline

For the supervised model, a Random Forest Classifier has been used on both datasets resulted from feature extraction.

Below are 2 tables 2 3 displaying the validation accuracies on both datasets obtained during hyperparameter tuning phase of the random forest. The model with the highest accuracy on validation set was kept.

For the TF-IDF features, the model with highest accuracy (43.1%) used a max depth of 100 and an ensemble size of 200. For the Fasttext features, the model with highest accuracy (55.7%) used a max depth of 50 and an ensemble size of 200.

Table 2: Hyperparameter Tuning of Random Forest on TF-IDF Features

Max Depth \ Num Estimators	10	20	50	70	100	200
5	36%	36.8%	36.9%	36.9%	37.2%	37.2%
10	37.8%	38.3%	38.3%	38.4%	38.5%	38.5%
15	38.6%	38.9%	38.8%	38.8%	38.8%	38.6%
20	39.4%	39.3%	39.2%	39.1%	39.1%	39.1%
30	40.6%	40.6%	40.8%	40.7%	40.6%	40.8%
50	41.7%	42%	42%	41.9%	42%	42.2%
70	41.6%	42.1%	42.6%	42.8%	42.8%	42.9%
100	41.7%	42.1%	42.7%	42.7%	42.7%	**43.1%**
200	41.7%	42.1%	42.8%	42.7%	42.9%	43.1%
None	41.7%	41.9%	42.6%	42.6%	42.9%	43%

Table 3: Hyperparameter Tuning of Random Forest on Fasttext Features

Max Depth \ Num Estimators	10	20	50	70	100	200
5	38%	37.2%	36.7%	36.7%	36.9%	36.8%
10	46%	46.5%	46.4%	46.3%	46.2%	46.4%
15	49.8%	52.4%	53.7%	54.1%	54.1%	54.2%
20	49.1%	51.9%	53.8%	54.5%	55.2%	55.3%
30	49.5%	52%	54.5%	54.6%	55.3%	55.7%
50	48.9%	51.7%	54.5%	54.8%	55.5%	**55.7%**
70	48.9%	51.7%	54.5%	54.8%	55.5%	55.7%
100	48.9%	51.7%	54.5%	54.8%	55.5%	55.7%
200	48.9%	51.7%	54.5%	54.8%	55.5%	55.7%
None	48.9%	51.7%	54.5%	54.8%	55.5%	55.7%

Hyperparameter Tuning of Unsupervised Methods

During training, both unsupervised methods have the final number of generated clusters set to 8 (the number of different sentiments in the dataset).

The Hungarian algorithm have been applied on the negative normalized confusion matrix in order to obtain the best accuracy mapping between cluster ids and real data labels. The confusion matrix has been normalized by dividing each cell with the sum on that row, in order to take into account the unbalanced nature of the dataset. The model with the greatest accuracy on the validation set has been kept for each method of unsupervised learning.

Below are 4 tables 4 5 6 7 displaying the validation accuracies, validation normalized mutual information (NMI) and validation silhouette score (in this order) on both datasets obtained during hyperparameter tuning phase for both BIRCH and Fuzzy C-Means methods. The model with the highest accuracy on validation set was kept.

Table 4: Hyperparameter Tuning of BIRCH on TF-IDF Features

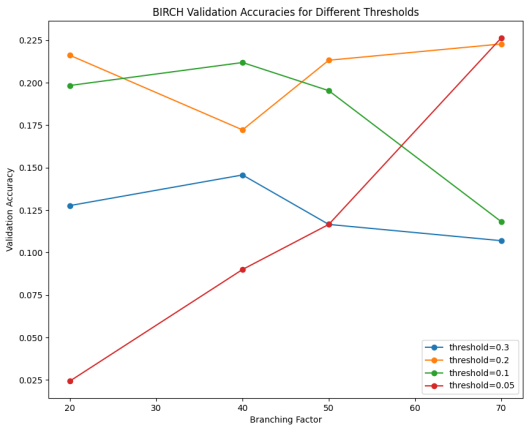
Threshold \ Branching Factor	20	40	50	70
0.3	12.7% 0.021 0.033	14.5% 0.016 0.075	11.6% 0.019 0.031	10.7% 0.018 0.043
0.2	21.6% 0.024 0.047	17.2% 0.02 0.071	21.3% 0.022 -0.022	22.2% 0.02 0.056
0.1	19.8% 0.021 0.059	21.1% 0.027 -0.027	19.5% 0.021 0.066	11.8% 0.021 0.047
0.05	2.44% 0.013 0.081	9% 0.015 0.076	11.6% 0.02 0.058	**22.6%** 0.022 -0.021

Table 5: Hyperparameter Tuning of Fuzzy C-Means on TF-IDF Features

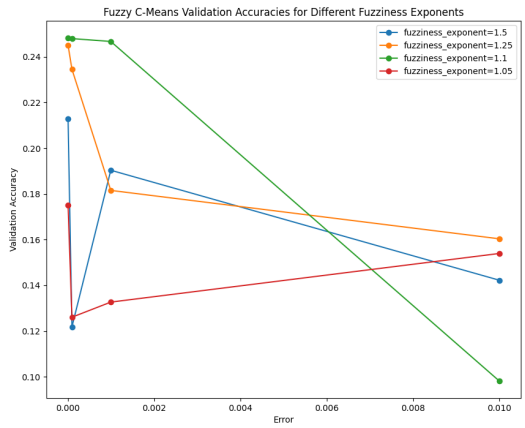
Fuzziness Exponent \ Error	1e-2	1e-3	1e-4	1e-5
1.5	14.2% 0.017 0.031	19% 0.018 0.033	12.1% 0.019 0.025	21.2% 0.018 0.03
1.25	16% 0.019 0.047	18.1% 0.02 0.048	23.4% 0.019 0.046	24.5% 0.018 0.037
1.1	9.81% 0.02 0.051	24.6% 0.017 0.049	24.7% 0.015 0.045	**24.8%** 0.016 0.045
1.05	15.3% 0.025 0.055	13.2% 0.023 0.061	12.6% 0.023 0.055	17.5% 0.023 0.055

For the TF-IDF features, the best BIRCH (22.6%) used a threshold of 0.05 and a maximum branching factor of 70, while the best Fuzzy C-Means (24.8%) used a fuzziness exponent of 1.1 and an error of 1e-5.

Below are 2 plots 3a 3b showing how validation accuracies evolve during hyperparameter tuning phase of the two unsupervised methods on TF-IDF features.



(a) BIRCH TF-IDF Validation Accuracies



(b) Fuzzy C-Means TF-IDF Validation Accuracies

For the Fasttext features, the best BIRCH (10%) used a threshold of 0.01 and a maximum branching factor of 90, while the best Fuzzy C-Means (18.7%) used a fuzziness exponent of 1.1 and an error of 1e-3.

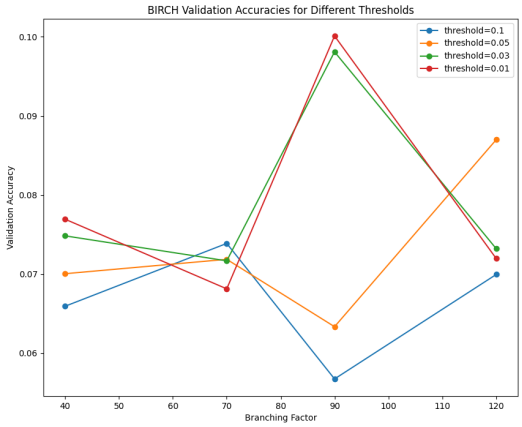
Below are 2 plots 4a 4b showing how validation accuracies evolve during hyperparameter tuning phase of the two unsupervised methods on Fasttext features.

Table 6: Hyperparameter Tuning of BIRCH on Fasttext Features

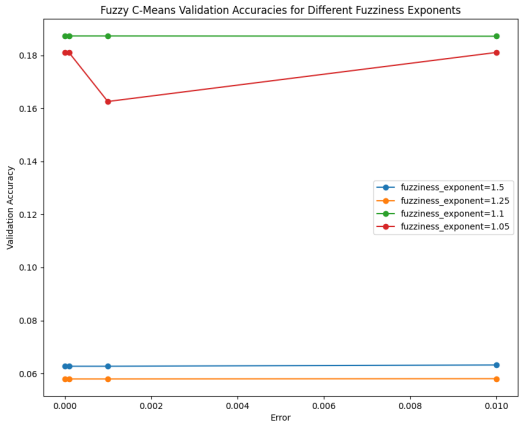
Threshold \ Branching Factor	40	70	90	120
0.1	6.59% 0.04 0.092	7.38% 0.036 0.042	5.67% 0.045 0.123	6.99% 0.036 0.043
0.05	7% 0.038 0.036	7.18% 0.038 0.037	6.33% 0.042 0.09	8.69% 0.039 0.051
0.03	7.48% 0.044 0.064	7.16% 0.038 0.032	9.81% 0.038 0.035	7.31% 0.038 0.068
0.01	7.69% 0.036 0.052	6.81% 0.037 0.05	**10%** 0.036 0.029	7.19% 0.039 0.035

Table 7: Hyperparameter Tuning of Fuzzy C-Means on Fasttext Features

Fuzziness Exponent \ Error	1e-2	1e-3	1e-4	1e-5
1.5	6.32% 0.013 -0.07	6.27% 0.012 -0.107	6.27% 0.012 0.049	6.27% 0.012 0.22
1.25	5.8% 0.025 -0.091	5.79% 0.025 0.048	5.79% 0.025 0.048	5.79% 0.025 0.048
1.1	18.7% 0.032 0.014	**18.7%** 0.032 0.014	18.7% 0.032 0.014	18.7% 0.032 0.014
1.05	18.1% 0.039 0.032	16.2% 0.041 0.035	18.1% 0.039 0.032	18.1% 0.039 0.032



(a) BIRCH Fasttext Validation Accuracies



(b) Fuzzy C-Means Fasttext Validation Accuracies

Final Results

For random chance prediction, the obtained validation accuracies for both TF-IDF and Fasttext were 12.3% and 12.2%, respectively (which are approximately the theoretical accuracy of $\frac{1}{8} = 0.125$, where 8 is the number of classes in the dataset). We will use 12.5% as the random chance baseline.

Below is the table 8 with best final validation accuracies for the two unsupervised methods, along with the supervised baseline and the random chance.

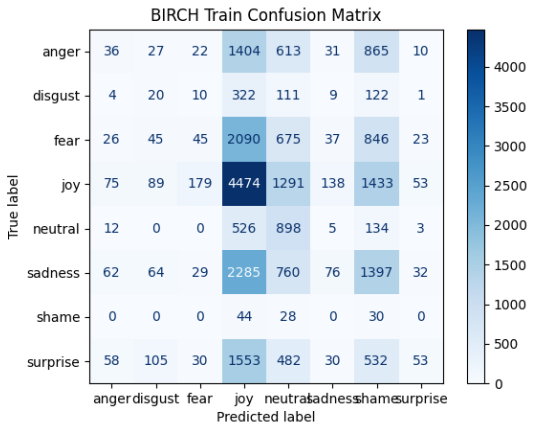
Table 8: Final Validation Accuracies

Features \ Model	Random Chance	Random Forest	BIRCH	Fuzzy C-Means
TF-IDF	12.5%	43.1%	22.6%	24.8%
Fasttext	12.5%	55.7%	10%	18.7%

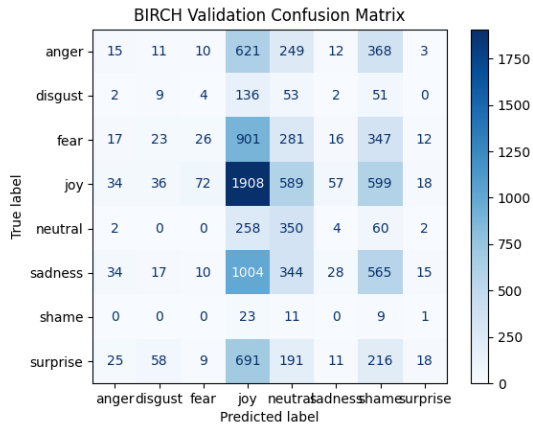
Visualizations and Interpretations

TF-IDF Features

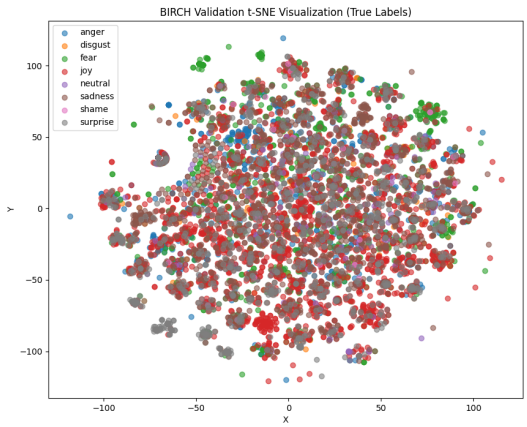
BIRCH



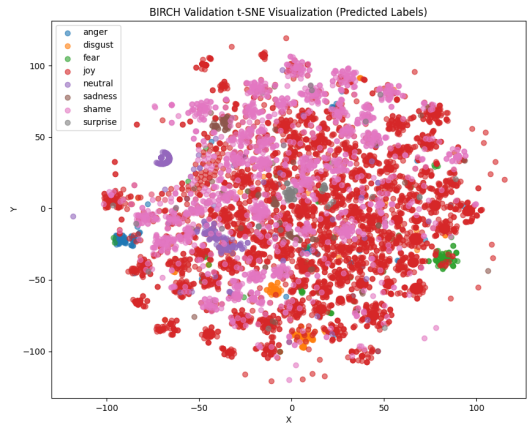
(a) BIRCH TF-IDF Train Confusion Matrix



(b) BIRCH TF-IDF Validation Confusion Matrix



(a) BIRCH TF-IDF Validation t-SNE True Labels



(b) BIRCH TF-IDF Validation t-SNE Predicted Labels

Table 9: BIRCH TF-IDF Train and Validation Precisions and Recalls per Class

Sentiment	anger	disgust	fear	joy	neutral	sadness	shame	surprise
Train Precision	13.1%	5.71%	14.2%	35.2%	18.4%	23.3%	0.559%	30.2%
Validation Precision	11.6%	5.84%	19.8%	34.4%	16.9%	21.5%	0.406%	26%
Train Recall	1.19%	3.33%	1.18%	57.8%	56.9%	1.61%	29.4%	1.86%
Validation Recall	1.16%	3.5%	1.6%	57.5%	51.7%	1.38%	20.4%	1.47%

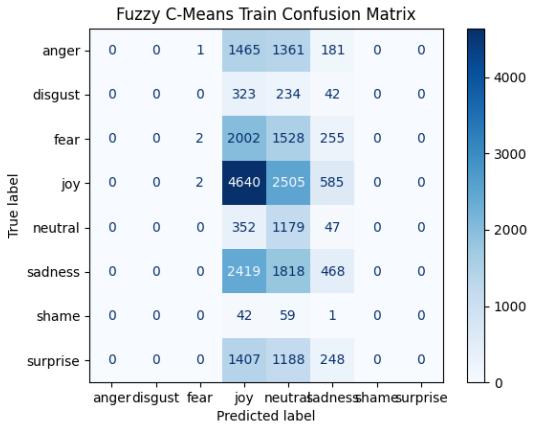
BIRCH on TF-IDF had a Train F1-Score of 17.4% and a Validation F1-Score of 16.9%.

Below is a table 10 showing what were the most influential words for each separate class (results taken on BIRCH's predicted labels for the validation TF-IDF set).

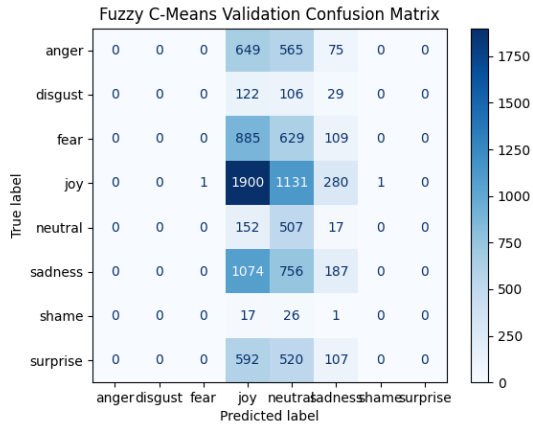
Table 10: Top 10 Most Influential Words per Sentiment (BIRCH TF-IDF Validation)

	anger	disgust	fear	joy	neutral	sadness	shame	surprise
1.	cant (0.461)	rt (0.391)	amp (0.665)	im (0.041)	hours (0.026)	year (0.331)	night (0.042)	coming (0.637)
2.	wait (0.385)	lol (0.361)	god (0.048)	time (0.035)	sleep (0.018)	old (0.331)	see (0.039)	home (0.107)
3.	im (0.054)	im (0.05)	never (0.032)	love (0.033)	im (0.003)	years (0.065)	friend (0.039)	back (0.088)
4.	tomorrow (0.053)	friends (0.037)	life (0.03)	dont (0.032)	day (0.003)	next (0.057)	felt (0.037)	family (0.045)
5.	see (0.047)	away (0.035)	love (0.028)	day (0.031)	work (0.003)	time (0.055)	could (0.034)	night (0.036)
6.	go (0.034)	one (0.031)	work (0.027)	like (0.03)	tomorrow (0.003)	first (0.045)	would (0.033)	im (0.035)
7.	get (0.03)	amp (0.029)	like (0.024)	today (0.029)	get (0.002)	school (0.036)	last (0.031)	feel (0.033)
8.	make (0.025)	dont (0.027)	need (0.023)	get (0.028)	go (0.002)	im (0.036)	one (0.031)	see (0.032)
9.	come (0.024)	get (0.025)	day (0.023)	people (0.026)	going (0.002)	friend (0.03)	face (0.03)	like (0.028)
10.	back (0.024)	know (0.024)	feel (0.021)	tomorrow (0.025)	school (0.002)	going (0.027)	house (0.03)	didnt (0.028)

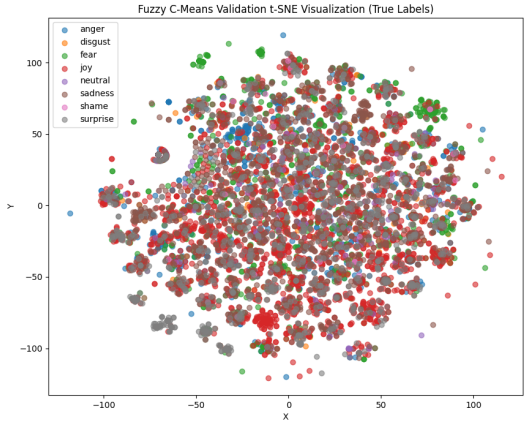
Fuzzy C-Means



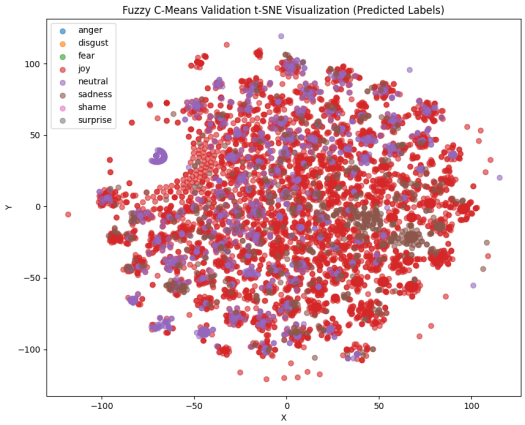
(a) Fuzzy C-Means TF-IDF Train Confusion Matrix



(b) Fuzzy C-Means TF-IDF Validation Confusion Matrix



(a) Fuzzy C-Means TF-IDF Validation t-SNE True Labels



(b) Fuzzy C-Means TF-IDF Validation t-SNE Predicted Labels

Fuzzy C-Means on TF-IDF had a Train F1-Score of 18.5% and a Validation F1-Score of 17.7%.

Below is a table 12 showing what were the most influential words for each separate class (results taken on Fuzzy C-Means' predicted labels for the validation TF-IDF set).

Table 11: Fuzzy C-Means TF-IDF Train and Validation Precisions and Recalls per Class

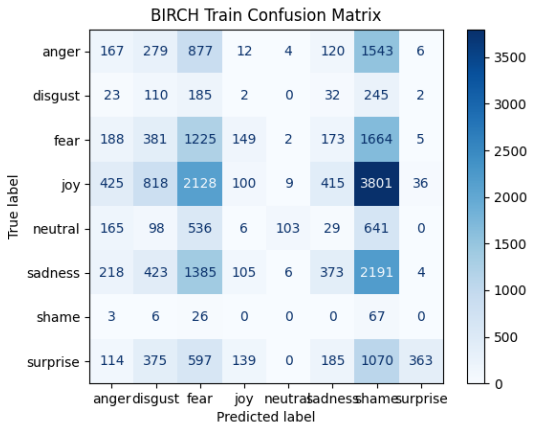
Sentiment	anger	disgust	fear	joy	neutral	sadness	shame	surprise
Train Precision	0%	0%	40%	36.6%	11.9%	25.6%	0%	0%
Validation Precision	0%	0%	0%	35.2%	11.9%	23.2%	0%	0%
Train Recall	0%	0%	0.052%	60%	74.7%	9.94%	0%	0%
Validation Recall	0%	0%	0%	57.3%	75%	9.27%	0%	0%

Table 12: Top 10 Most Influential Words per Sentiment (Fuzzy C-Means TF-IDF Validation)

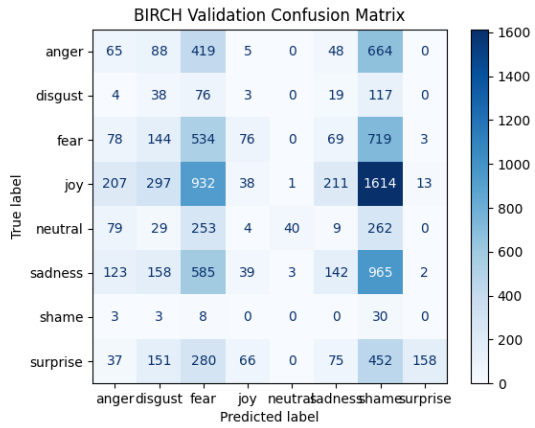
	anger	disgust	fear	joy	neutral	sadness	shame	surprise
1.	-	-	hours (0.615)	time (0.046)	de (0.016)	im (0.376)	ill (0.441)	-
2.	-	-	long (0.596)	day (0.041)	en (0.015)	going (0.161)	god (0.441)	-
3.	-	-	going (0.514)	love (0.039)	yes (0.014)	dont (0.089)	tonight (0.438)	-
4.	-	-	yesterday (0)	like (0.036)	een (0.013)	think (0.031)	night (0.38)	-
5.	-	-	getting (0)	one (0.035)	face (0.009)	get (0.03)	going (0.377)	-
6.	-	-	god (0)	get (0.034)	fear (0.009)	tomorrow (0.03)	tomorrow (0.361)	-
7.	-	-	go (0)	today (0.034)	felt (0.009)	gonna (0.029)	girl (0)	-
8.	-	-	give (0)	back (0.031)	little (0.009)	know (0.028)	go (0)	-
9.	-	-	girl (0)	people (0.03)	right (0.008)	like (0.028)	give (0)	-
10.	-	-	get (0)	got (0.029)	look (0.008)	cant (0.028)	yesterday (0)	-

Fasttext Features

BIRCH



(a) BIRCH Fasttext Train Confusion Matrix



(b) BIRCH Fasttext Validation Confusion Matrix

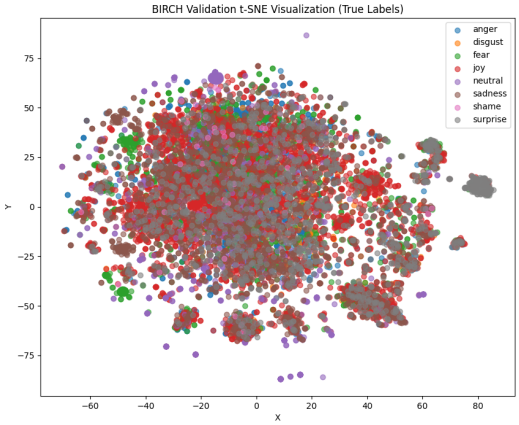
Table 13: BIRCH Fasttext Train and Validation Precisions and Recalls per Class

Sentiment	anger	disgust	fear	joy	neutral	sadness	shame	surprise
Train Precision	12.8%	4.41%	17.6%	19.4%	83%	28.1%	0.597%	87.2%
Validation Precision	10.9%	4.18%	17.2%	16.4%	9.09%	24.7%	0.622%	89.7%
Train Recall	5.55%	18.3%	32.3%	1.29%	6.52%	7.92%	65.6%	12.7%
Validation Recall	5.04%	14.7%	32.9%	1.14%	5.91%	7.04%	68.1%	12.9%

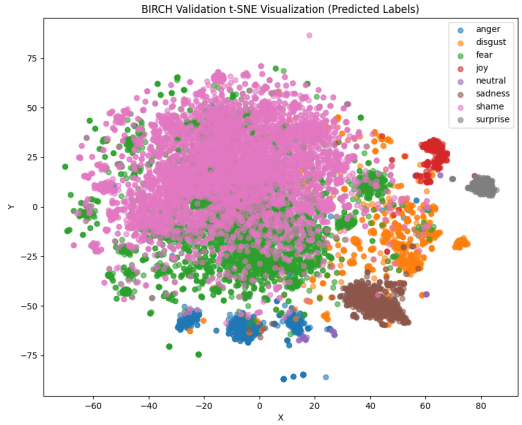
BIRCH on Fasttext had a Train F1-Score of 11.2% and a Validation F1-Score of 10.7%.

Fuzzy C-Means

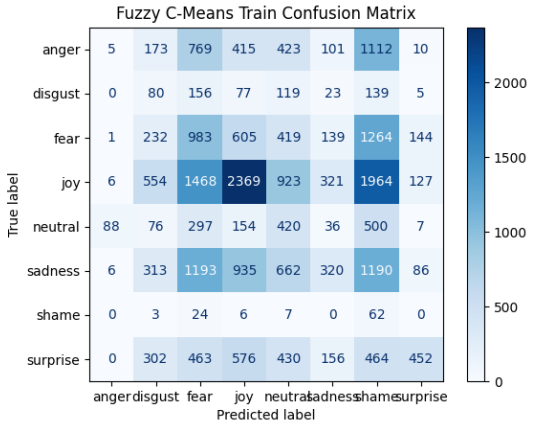
Fuzzy C-Means on Fasttext had a Train F1-Score of 21.3% and a Validation F1-Score of 20.5%.



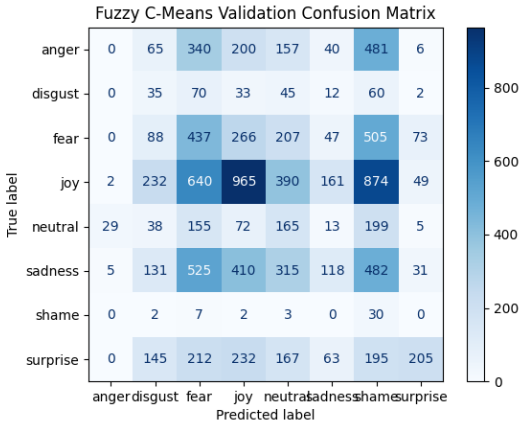
(a) BIRCH Fasttext Validation t-SNE True Labels



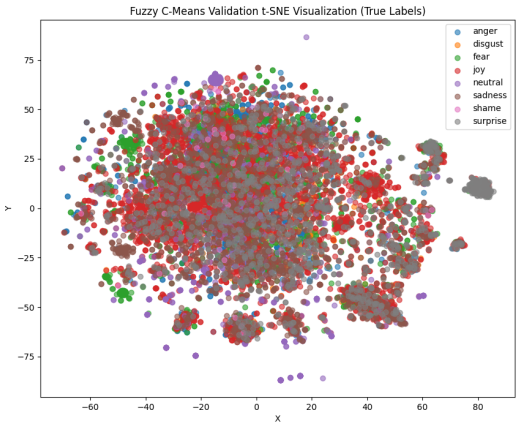
(b) BIRCH Fasttext Validation t-SNE Predicted Labels



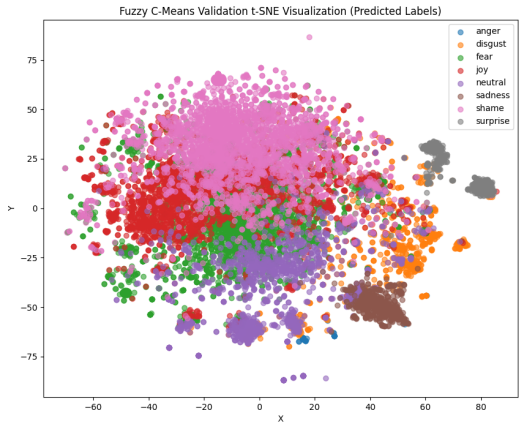
(a) Fuzzy C-Means Fasttext Train Confusion Matrix



(b) Fuzzy C-Means Fasttext Validation Confusion Matrix



(a) Fuzzy C-Means Fasttext Validation t-SNE True Labels



(b) Fuzzy C-Means Fasttext Validation t-SNE Predicted Labels

Table 14: Fuzzy C-Means Fasttext Train and Validation Precisions and Recalls per Class

Sentiment	anger	disgust	fear	joy	neutral	sadness	shame	surprise
Train Precision	4.71%	4.61%	18.3%	46.1%	12.3%	29.1%	0.926%	54.3%
Validation Precision	0%	4.75%	18.3%	44.2%	11.3%	25.9%	1.06%	55.2%
Train Recall	0.166%	13.3%	25.9%	30.6%	26.6%	6.8%	60.7%	15.8%
Validation Recall	0%	13.6%	26.9%	29.1%	24.4%	5.85%	68.1%	16.8%

Conclusions

Both unsupervised approaches have achieved a lower validation accuracy than the supervised baseline, but higher than the random choice one (the only exception being BIRCH trained on Fasttext features, which achieved only 10% maximum accuracy and could not surpass random choice).

Fuzzy C-Means achieved greater performance than BIRCH on both feature sets. However, looking at precision and recalls values, we can see that Fuzzy C-Means was not very robust to an unbalanced dataset (sentiments with few samples, such as anger, shame, disgust and surprise, were ignored in favour of more frequent classes, like joy). BIRCH, on the other hand, handled better the unbalanced dataset, even if the overall accuracy was lower (the precisions and recalls per class were more uniformly distributed, whereas for Fuzzy C-Means they concentrated on classes with many samples).

Both unsupervised methods performed better on less complex features (TF-IDF with 128 features, rather than Fasttext with 300 features).